Experiment Question: For a given set of training examples stored in a CSV file, implement the Candidate-Elimination algorithm in Python to output a description of the set of all hypotheses consistent with the training examples.

Formatted: Centered

Subproblem 1: Enjoy Sport / Weather Dataset

1. Aim

To implement Candidate-Elimination using a CSV file and calculate the specific and general boundaries for the Enjoy Sport concept.

2. Concept

Each row describes Sky, Temperature, Humidity, and Wind. The target is Enjoy. Candidate-Elimination updates S and G after each positive or negative example.

3. Dataset Description

The following table is also written to a CSV file by the program.

Sky	Temperature	Humidity	Wind	Enjoy
Sunny	Warm	Normal	Strong	Yes
Sunny	Warm	High	Strong	Yes
Rainy	Cold	High	Strong	No
Sunny	Warm	High	Weak	Yes

4. Algorithm

- Read the CSV file and separate attributes from the target column.
- Find the observed domain values for each attribute.
- Initialize S to the most-specific hypothesis and G to the most-general hypothesis.
- For a positive example, remove G hypotheses that do not cover the example and minimally generalize S.
- For a negative example, remove S hypotheses that cover the example and minimally specialize G.
- Prune hypotheses that are not on the relevant boundary and display S and G.

5. Step-by-step Procedure and Detailed Mathematical Calculations

- Initialize $S_0 = \{(Ø, Ø, Ø, Ø)\}$ and $G_0 = \{(?, ?, ?, ?)\}$.
- Example 1 (positive): x = (Sunny, Warm, Normal, Strong). S generalizes to (Sunny, Warm, Normal, Strong); G still covers x, so G remains (?, ?, ?, ?).
- Example 2 (positive): x = (Sunny, Warm, High, Strong). S differs only at Humidity, so S becomes (Sunny, Warm, ?, Strong). G remains (?, ?, ?, ?).
- Example 3 (negative): x = (Rainy, Cold, High, Strong). S does not cover this negative example, so S stays (Sunny, Warm, ?, Strong). G covers it, so specialize G minimally using observed domains: (Sunny, ?, ?, ?) and (?, Warm, ?, ?) are useful specializations that exclude this negative example and remain at least as general as S. Other one-attribute specializations do not remain compatible with S.

- Example 4 (positive): x = (Sunny, Warm, High, Weak). Remove any G hypotheses that fail to cover this positive example. (?, Warm, ?, ?) covers it; (Sunny, ?, ?, ?) covers it. S generalizes Wind, giving S = (Sunny, Warm, ?, ?). Final boundaries: S = {(Sunny, Warm, ?, ?)}; G = {(Sunny, ?, ?, ?), (?, Warm, ?, ?)}.

6. Formulae

Hypothesis coverage: covers(h, x) = True iff for every i, h_i = ? or h_i = x_i.

Consistency: h(x) = y for every training pair (x, y).

Positive update: S ← minimal generalizations of S that cover x; G ← {g ∈ G : covers(g, x)}.

Negative update: S ← {s ∈ S : not covers(s, x)}; G ← minimal specializations of G that do not cover x.

Specific boundary: S = {(Sunny, Warm, ?, ?)}.

General boundary: G = {(Sunny, ?, ?, ?) and (?, Warm, ?, ?)}.

Calculation note: this algorithm uses symbolic comparisons rather than arithmetic sums. The detailed calculation is the attribute-by-attribute coverage and boundary update for every row.

7. Python Program

Run the shared Candidate-Elimination function from the “Common Python Implementation” section first. Then run the following subproblem code to create the CSV file and call the function.

```
import csv
# Save this data as sport.csv, then run the shared candidate_elimination
# implementation from the beginning of this experiment.
rows = [
    ['Sky', 'Temperature', 'Humidity', 'Wind', 'Enjoy'],
    ['Sunny', 'Warm', 'Normal', 'Strong', 'Yes'],
    ['Sunny', 'Warm', 'High', 'Strong', 'Yes'],
    ['Rainy', 'Cold', 'High', 'Strong', 'No'],
    ['Sunny', 'Warm', 'High', 'Weak', 'Yes']
]
with open('sport.csv', 'w', newline='') as f:
    csv.writer(f).writerows(rows)

candidate_elimination('sport.csv')
```

8. Output

```
>>>
====== RESTART: C:/Users/SREE/AppData/Local/Programs/Python/Python313/2.py =====
Specific Boundary S:
('Sunny', 'Warm', '?', '?')
General Boundary G:
('?', 'Warm', '?', '?')
('Sunny', '?', '?', '?')
>>>
```

9. Result / Conclusion

Candidate-Elimination was demonstrated on this dataset. The specific boundary is S = {(Sunny, Warm, ?, ?)} and the general boundary is G = {(Sunny, ?, ?, ?) and (?, Warm, ?, ?)}. These boundaries summarize the hypotheses consistent with the examples under the stated categorical hypothesis representation.

Subproblem 2: Student Performance Dataset

1. Aim

To find all boundary hypotheses consistent with examples describing attendance, internal marks, assignment submission, and passing status.

2. Concept

The input attributes are Attendance, Internal, and Assignment. Pass is encoded as Yes and Fail as No.

3. Dataset Description

The following table is also written to a CSV file by the program.

Attendance	Internal	Assignment	Pass
High	Good	Regular	Yes
High	Good	Irregular	Yes
Low	Poor	Regular	No
High	Good	Regular	Yes

4. Algorithm

- Read the CSV file and separate attributes from the target column.
- Find the observed domain values for each attribute.
- Initialize S to the most-specific hypothesis and G to the most-general hypothesis.
- For a positive example, remove G hypotheses that do not cover the example and minimally generalize S.
- For a negative example, remove S hypotheses that cover the example and minimally specialize G.
- Prune hypotheses that are not on the relevant boundary and display S and G.

- 5. Step-by-step Procedure and Detailed Mathematical Calculations
  - Initialize $S_0 = \{(Ø, Ø, Ø)\}$ and $G_0 = \{(?, ?, ?)\}$.
  - Example 1 positive: (High, Good, Regular). S_1 = (High, Good, Regular); G remains (?, ?, ?).
  - Example 2 positive: (High, Good, Irregular). Assignment differs, so S_2 = (High, Good, ?).
  - Example 3 negative: (Low, Poor, Regular). S does not cover this negative example, so S remains (High, Good, ?). G covers it; minimally specialize G. Candidate specializations consistent with S are (High, ?, ?) and (?, Good, ?).
  - Example 4 positive: (High, Good, Regular). Both S and the two G hypotheses cover the example, so boundaries do not change. Final S = {(High, Good, ?)}; G = {(High, ?, ?), (?, Good, ?)}.

6. Formulae

Hypothesis coverage: covers(h, x) = True iff for every i, h_i = ? or $h_i = x_i$.

Consistency: h(x) = y for every training pair (x, y).

Positive update: S ← minimal generalizations of S that cover x; G ← {g ∈ G : covers(g, x)}.

Negative update: S ← {s ∈ S : not covers(s, x)}; G ← minimal specializations of G that do not cover x.

Specific boundary: S = {(High, Good, ?)}.

General boundary: G = {(High, ?, ?) and (?, Good, ?)}.

Calculation note: this algorithm uses symbolic comparisons rather than arithmetic sums. The detailed calculation is the attribute-by-attribute coverage and boundary update for every row.

7. Python Program

Run the shared Candidate-Elimination function from the “Common Python Implementation” section first. Then run the following subproblem code to create the CSV file and call the function.

```
import csv
rows = [
    ['Attendance', 'Internal', 'Assignment', 'Pass'],
    ['High', 'Good', 'Regular', 'Yes'],
    ['High', 'Good', 'Irregular', 'Yes'],
    ['Low', 'Poor', 'Regular', 'No'],
    ['High', 'Good', 'Regular', 'Yes']
]
with open('student.csv', 'w', newline='') as f:
    csv.writer(f).writerows(rows)

candidate_elimination('student.csv')
```

8. Output

```
>>>
====== RESTART: C:/Users/SREE/AppData/Local/Programs/Python/Python313/2.py =====
Specific Boundary S:
('High', 'Good', '?')
General Boundary G:
('?', 'Good', '?')
('High', '?', '?')
>>>
```

9. Result / Conclusion

Candidate-Elimination was demonstrated on this dataset. The specific boundary is S = {(High, Good, ?)} and the general boundary is G = {(High, ?, ?) and (?, Good, ?)}. These boundaries summarize the hypotheses consistent with the examples under the stated categorical hypothesis representation.

Subproblem 3: Computer Purchase Dataset

1. Aim

To implement Candidate-Elimination for predicting purchase using price, speed, and memory attributes.

2. Concept

Price, Speed, and Memory are categorical input features. Purchase is the target label.

3. Dataset Description

The following table is also written to a CSV file by the program.

Price	Speed	Memory	Purchase
High	Fast	Large	Yes
High	Fast	Small	Yes
Low	Slow	Small	No
Medium	Fast	Large	Yes

4. Algorithm

- Read the CSV file and separate attributes from the target column.
- Find the observed domain values for each attribute.
- Initialize S to the most-specific hypothesis and G to the most-general hypothesis.
- For a positive example, remove G hypotheses that do not cover the example and minimally generalize S.
- For a negative example, remove S hypotheses that cover the example and minimally specialize G.
- Prune hypotheses that are not on the relevant boundary and display S and G.

5. Step-by-step Procedure and Detailed Mathematical Calculations

- Initialize $S_0 = \{(Ø, Ø, Ø)\}$ and $G_0 = \{(?, ?, ?)\}$.
- Example 1 positive: (High, Fast, Large). S_1 = (High, Fast, Large).
- Example 2 positive: (High, Fast, Small). Memory differs, so S_2 = (High, Fast, ?).
- Example 3 negative: (Low, Slow, Small). S does not cover the negative example. G covers it, so minimally specialize G with observed domains. Specializations compatible with S include (High, ?, ?) and (?, Fast, ?).
- Example 4 positive: (Medium, Fast, Large). S generalizes Price because High ≠ Medium, giving S_4 = (?, Fast, ?). The G hypothesis (High, ?, ?) does not cover this positive example and is removed. (?, Fast, ?) remains. Final S = {(?, Fast, ?)}; G = {(?, Fast, ?)}.

6. Formulae

Hypothesis coverage: covers(h, x) = True iff for every i, h_i = ? or $h_i = x_i$.

Consistency: h(x) = y for every training pair (x, y).

Positive update: S ← minimal generalizations of S that cover x; G ← {g ∈ G : covers(g, x)}.

Negative update: S ← {s ∈ S : not covers(s, x)}; G ← minimal specializations of G that do not cover x.

Specific boundary: S = {(?, Fast, ?)}.

General boundary: G = {(?, Fast, ?)}.

Calculation note: this algorithm uses symbolic comparisons rather than arithmetic sums. The detailed calculation is the attribute-by-attribute coverage and boundary update for every row.

7. Python Program

Run the shared Candidate-Elimination function from the “Common Python Implementation” section first. Then run the following subproblem code to create the CSV file and call the function.

```
import csv
rows = [
    ['Price', 'Speed', 'Memory', 'Purchase'],
    ['High', 'Fast', 'Large', 'Yes'],
    ['High', 'Fast', 'Small', 'Yes'],
    ['Low', 'Slow', 'Small', 'No'],
    ['Medium', 'Fast', 'Large', 'Yes']
]
with open('computer.csv', 'w', newline='') as f:
    csv.writer(f).writerows(rows)

candidate_elimination('computer.csv')
```

8. Output

```
>>>
====== RESTART: C:/Users/SREE/AppData/Local/Programs/Python/Python313/2.py =====
Specific Boundary S:
('?', 'Fast', '?')
General Boundary G:
('?', 'Fast', '?')
>>>
```

9. Result / Conclusion

Candidate-Elimination was demonstrated on this dataset. The specific boundary is S = {(?, Fast, ?)} and the general boundary is G = {(?, Fast, ?)}. These boundaries summarize the hypotheses consistent with the examples under the stated categorical hypothesis representation.

Subproblem 4: Animal Category Dataset

1. Aim

To determine consistent hypotheses for an animal category from body temperature, number of legs, and habitat.

2. Concept

The categorical features are Body Temperature, Legs, and Habitat. The class Animal is Yes or No.

3. Dataset Description

The following table is also written to a CSV file by the program.

Body	Legs	Habitat	Animal
Warm	Four	Land	Yes
Warm	Four	Water	Yes
Cold	Two	Water	No
Warm	Four	Land	Yes

4. Algorithm

- Read the CSV file and separate attributes from the target column.
- Find the observed domain values for each attribute.
- Initialize S to the most-specific hypothesis and G to the most-general hypothesis.
- For a positive example, remove G hypotheses that do not cover the example and minimally generalize S.
- For a negative example, remove S hypotheses that cover the example and minimally specialize G.
- Prune hypotheses that are not on the relevant boundary and display S and G.

- 5. Step-by-step Procedure and Detailed Mathematical Calculations
  - Initialize $S_0 = \{(Ø, Ø, Ø)\}$ and $G_0 = \{(?, ?, ?)\}$.
  - Example 1 positive: (Warm, Four, Land). S_1 = (Warm, Four, Land).
  - Example 2 positive: (Warm, Four, Water). Habitat differs, so S_2 = (Warm, Four, ?).
  - Example 3 negative: (Cold, Two, Water). S does not cover the negative example. Specializing G gives candidate hypotheses including (Warm, ?, ?) and (?, Four, ?), both consistent with S and excluding the negative example.
  - Example 4 positive: (Warm, Four, Land). Both candidate G hypotheses cover it. Final S = {(Warm, Four, ?)}; G = {(Warm, ?, ?), (?, Four, ?)}.

6. Formulae

Hypothesis coverage: covers(h, x) = True iff for every i, $h_i = ?$ or $h_i = x_i$.

Consistency: h(x) = y for every training pair (x, y).

Positive update: S ← minimal generalizations of S that cover x; G ← $\{g \in G : \text{covers}(g, x)\}$.

Negative update: S ← {s ∈ S : not covers(s, x)}; G ← minimal specializations of G that do not cover x.

Specific boundary: S = {(Warm, Four, ?)}.

General boundary: G = {(Warm, ?, ?) and (?, Four, ?)}.

Calculation note: this algorithm uses symbolic comparisons rather than arithmetic sums. The detailed calculation is the attribute-by-attribute coverage and boundary update for every row.

7. Python Program

Run the shared Candidate-Elimination function from the “Common Python Implementation” section first. Then run the following subproblem code to create the CSV file and call the function.

```
import csv
rows = [
    ['Body', 'Legs', 'Habitat', 'Animal'],
    ['Warm', 'Four', 'Land', 'Yes'],
    ['Warm', 'Four', 'Water', 'Yes'],
    ['Cold', 'Two', 'Water', 'No'],
    ['Warm', 'Four', 'Land', 'Yes']
]
with open('animal.csv', 'w', newline='') as f:
    csv.writer(f).writerows(rows)

candidate_elimination('animal.csv')
```

8. Output

```
>>>
====== RESTART: C:/Users/SREE/AppData/Local/Programs/Python/Python313/2.py =====
Specific Boundary S:
('Warm', 'Four', '?')
General Boundary G:
('?', 'Four', '?')
('Warm', '?', '?')
>>>
```

9. Result / Conclusion

Candidate-Elimination was demonstrated on this dataset. The specific boundary is S = {(Warm, Four, ?)} and the general boundary is G = {(Warm, ?, ?) and (?, Four, ?)}. These boundaries summarize the hypotheses consistent with the examples under the stated categorical hypothesis representation.

Subproblem 5: Weather Suitability Dataset

1. Aim

To calculate the version-space boundaries for a weather suitability concept using Sky, Temperature, and Humidity.

2. Concept

This dataset contains three categorical attributes and a binary target Suitable. The dataset includes positive and negative examples.

3. Dataset Description

The following table is also written to a CSV file by the program.

Sky	Temperature	Humidity	Suitable
Sunny	Warm	Normal	Yes
Sunny	Warm	High	Yes
Rainy	Cold	High	No
Sunny	Warm	High	Yes

4. Algorithm

1. Read the CSV file and separate attributes from the target column.
2. Find the observed domain values for each attribute.
3. Initialize S to the most-specific hypothesis and G to the most-general hypothesis.
4. For a positive example, remove G hypotheses that do not cover the example and minimally generalize S.
5. For a negative example, remove S hypotheses that cover the example and minimally specialize G.
6. Prune hypotheses that are not on the relevant boundary and display S and G.

5. Step-by-step Procedure and Detailed Mathematical Calculations

7. Initialize $S_0 = \{(\varnothing, \varnothing, \varnothing)\}$ and $G_0 = \{(?, ?, ?)\}$.
8. Example 1 positive: (Sunny, Warm, Normal). S_1 = (Sunny, Warm, Normal).
9. Example 2 positive: (Sunny, Warm, High). Humidity differs, so S_2 = (Sunny, Warm, ?).
10. Example 3 negative: (Rainy, Cold, High). S does not cover it. Minimal specializations of G that remain compatible with S include (Sunny, ?, ?) and (?, Warm, ?).
11. Example 4 positive: (Sunny, Warm, High). Both G hypotheses cover the positive example. Final S = {(Sunny, Warm, ?)}; G = {(Sunny, ?, ?), (?, Warm, ?)}.

6. Formulae

Hypothesis coverage: covers(h, x) = True iff for every i, $h_i = ?$ or $h_i = x_i$.

Consistency: h(x) = y for every training pair (x, y).

Positive update: S ← minimal generalizations of S that cover x; G ← $\{g \in G : \text{covers}(g, x)\}$.

Negative update: S ← $\{s \in S : \text{not covers}(s, x)\}$; G ← minimal specializations of G that do not cover x.

Specific boundary: S = {(Sunny, Warm, ?)}.

General boundary: G = {(Sunny, ?, ?) and (?, Warm, ?)}.

Calculation note: this algorithm uses symbolic comparisons rather than arithmetic sums. The detailed calculation is the attribute-by-attribute coverage and boundary update for every row.

7. Python Program

Run the shared Candidate-Elimination function from the “Common Python Implementation” section first. Then run the following subproblem code to create the CSV file and call the function.

```
import csv
rows = [
    ['Sky', 'Temperature', 'Humidity', 'Suitable'],
    ['Sunny', 'Warm', 'Normal', 'Yes'],
    ['Sunny', 'Warm', 'High', 'Yes'],
    ['Rainy', 'Cold', 'High', 'No'],
    ['Sunny', 'Warm', 'High', 'Yes']
]
with open('weather.csv', 'w', newline='') as f:
    csv.writer(f).writerows(rows)

candidate_elimination('weather.csv')
```

8. Output

```
>>>
====== RESTART: C:/Users/SREE/AppData/Local/Programs/Python/Python313/2.py =====
Specific Boundary S:
('Sunny', 'Warm', '?')
General Boundary G:
('?', 'Warm', '?')
('Sunny', '?', '?')
>>>
```

9. Result / Conclusion

Candidate-Elimination was demonstrated on this dataset. The specific boundary is S = {(Sunny, Warm, ?)} and the general boundary is G = {(Sunny, ?, ?) and (?, Warm, ?)}. These boundaries summarize the hypotheses consistent with the examples under the stated categorical hypothesis representation.

Common Python Implementation

```
import csv
from itertools import product

def candidate_elimination(filename):
    with open(filename, newline="") as f:
        rows = list(csv.reader(f))

    headers = rows[0]
    examples = rows[1:]
    X = [r[:-1] for r in examples]
    Y = [r[-1].strip() for r in examples]
    domains = [sorted(set(x[i] for x in X)) for i in range(len(headers) - 1)]
    n = len(domains)

    # Most-specific and most-general boundaries
    S = {tuple("Ø" for _ in range(n))}
    G = {tuple("?" for _ in range(n))}

    def covers(h, x):
        return all(h[i] == "?" or h[i] == x[i] for i in range(n))

    def more_general_or_equal(h1, h2):
        # True when h1 is at least as general as h2
        for a, b in zip(h1, h2):
            if a == "?":
                continue
            if a != b:
                return False
        return True

    def generalize_minimally(s, x):
        h = list(s)
        for i in range(n):
            if h[i] == "Ø":
                h[i] = x[i]
            elif h[i] != x[i]:
                h[i] = "?"
        return tuple(h)

    def specialize_minimally(g, x):
        candidates = set()
        for i in range(n):
            if g[i] == "?":
                for value in domains[i]:
```

```
            if value != x[i]:
                h = list(g)
                h[i] = value
                # Keep only specializations at least as general as
                # one current S hypothesis (when possible).
                if any(more_general_or_equal(h, s) for s in S):
                    candidates.add(tuple(h))
    return candidates

for x, y in zip(X, Y):
    if y.lower() == "yes":
        G = {g for g in G if covers(g, x)}
        new_S = set()
        for s in S:
            if covers(s, x):
                new_S.add(s)
            else:
                h = generalize_minimally(s, x)
                if any(more_general_or_equal(g, h) for g in G):
                    new_S.add(h)
        S = {
            s for s in new_S
            if not any(s != other and more_general_or_equal(s, other)
                       for other in new_S)
        }
    else:
        S = {s for s in S if not covers(s, x)}
        new_G = set()
        for g in G:
            if not covers(g, x):
                new_G.add(g)
            else:
                new_G.update(specialize_minimally(g, x))
        # Remove hypotheses that are less general than another candidate.
        G = {
            g for g in new_G
            if not any(g != other and more_general_or_equal(other, g)
                       for other in new_G)
        }

print("Specific Boundary S:")
for h in sorted(S):
    print(h)
print("General Boundary G:")
for h in sorted(G):
    print(h)
```

```
if not S and not G:
    print("Version space is empty: no hypothesis satisfies all examples.")
```

Overall Result of Experiment 2

The Candidate-Elimination algorithm was demonstrated with five CSV datasets. For each dataset, the program reads examples, updates the S and G boundaries, and prints the boundary hypotheses. S contains maximally specific consistent hypotheses; G contains maximally general consistent hypotheses.

Subproblem	Dataset	Specific Boundary S		General Boundary G
1	Enjoy Sport / Weather	(Sunny, Warm, ?, ?)		(Sunny, ?, ?, ?); (?, Warm, ?, ?)
2	Student Performance	(High, Good, ?)		(High, ?, ?); (?, Good, ?)
3	Computer Purchase	(?, Fast, ?)		(?, Fast, ?)
4	Animal Category	(Warm, Four, ?)		(Warm, ?, ?); (?, Four, ?)
5	Weather Suitability	(Sunny, Warm, ?)		(Sunny, ?, ?); (?, Warm, ?)

Final conclusion: Candidate-Elimination maintains a version space rather than selecting only one hypothesis. The result depends on the attribute domains and hypothesis representation. For a mathematically rigorous implementation, minimal specializations and boundary pruning must be applied carefully; inspect printed output when changing the data or attribute domains.